

Class Activity

1) Autoencoder

Definition

An **Autoencoder (AE)** is an unsupervised neural network that learns a compressed representation (latent vector) of input data and then reconstructs the original input from that compressed encoding.

Mathematically:

- **Encoder:**

$$z = f_{\theta}(x)$$

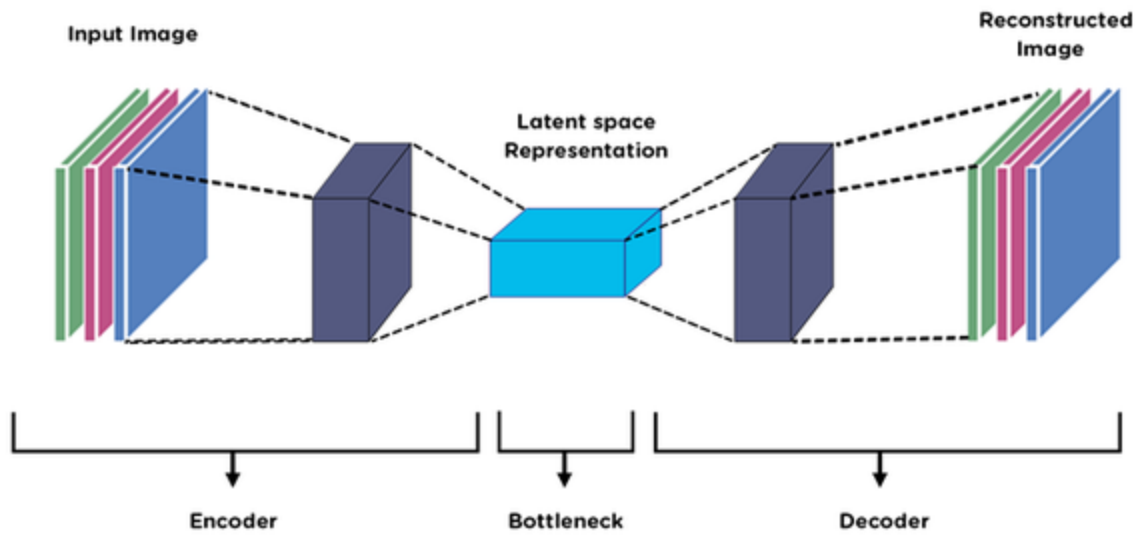
- **Decoder:**

$$\hat{x} = g_{\phi}(z)$$

- **Objective:** Minimize reconstruction loss

$$L = ||x - \hat{x}||^2 = ||x - g_{\phi}(f_{\theta}(x))||^2$$

It performs **dimensionality reduction + feature learning**.



Architecture Components:

1. **Input Layer (x)**
2. **Encoder**
 - Compresses input
 - Produces **latent representation (bottleneck layer)**
3. **Latent Space (z)**
 - Lowest dimensional representation
4. **Decoder**
 - Reconstructs input
5. **Output Layer ($\hat{x}$)**

Simple Application: MNIST Image Reconstruction

Since you're comfortable with deep learning (you've worked on CNN & YOLO-style models), here's a clean PyTorch implementation.

Use Case:

Reconstruct handwritten digits from MNIST.

PyTorch Implementation

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# Hyperparameters
batch_size = 128
latent_dim = 32
epochs = 5
lr = 1e-3

# Dataset
transform = transforms.ToTensor()
train_data = datasets.MNIST(root='./data', train=True, transform=transform, download=True)
train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True)

# Autoencoder Model
class Autoencoder(nn.Module):
    def __init__(self):
        super(Autoencoder, self).__init__()

        # Encoder
        self.encoder = nn.Sequential(
            nn.Linear(28*28, 128),
            nn.ReLU(),
            nn.Linear(128, latent_dim)
        )

        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 128),
```

```

        nn.ReLU(),
        nn.Linear(128, 28*28),
        nn.Sigmoid()
    )

    def forward(self, x):
        x = x.view(x.size(0), -1)
        z = self.encoder(x)
        out = self.decoder(z)
        return out

model = Autoencoder()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=lr)

# Training Loop
for epoch in range(epochs):
    total_loss = 0
    for images, _ in train_loader:
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, images.view(images.size(0), -1))
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    print(f"Epoch [{epoch+1}/{epochs}], Loss: {total_loss/len(train_loader):.4f}")

```

What This Model Does

- Compresses 784 → 32 dimensions
- Learns meaningful digit features
- Reconstructs digit images
- Useful for:
 - Denoising
 - Dimensionality reduction
 - Feature extraction

2) Variational Autoencoder (VAE)

Definition

A **Variational Autoencoder (VAE)** is a generative model that learns a probability distribution over latent space instead of a fixed encoding.

Instead of learning:

$$z = f(x)$$

It learns:

$$z \sim \mathcal{N}(\mu, \sigma^2)$$

The encoder outputs:

- Mean (μ)
- Log variance ($\log \sigma^2$)

Then sampling is done using:

$$z = \mu + \sigma \cdot \epsilon$$

where

$$\epsilon \sim \mathcal{N}(0, 1)$$

This is called the **reparameterization trick**.

VAE Loss Function

$$L = \text{Reconstruction Loss} + \text{KL Divergence}$$

$$L = \mathbb{E}[\|x - \hat{x}\|^2] + \text{KL}(q(z|x) || p(z))$$

Where:

- First term $\rightarrow$ reconstruction
- Second term $\rightarrow$ forces latent distribution to be Gaussian

Simple Example (Intuition)

Imagine generating handwritten digits:

- Normal AE → memorizes compression
- VAE → learns smooth latent space

If you sample random z from:

$$z \sim \mathcal{N}(0,1)$$

Decoder can generate **new digit images**.

This makes VAE a **generative model**, unlike vanilla AE.

3) Comparison: Autoencoder vs VAE

Feature	Autoencoder	VAE
Type	Deterministic	Probabilistic
Latent Space	Fixed vector	Distribution (μ, σ)
Sampling	Not possible	Yes
Generative	No	Yes
Loss Function	Reconstruction only	Reconstruction + KL Divergence
Latent Structure	Unstructured	Continuous & smooth
Use Cases	Compression, Denoising	Image generation, Data synthesis